

Go como lenguaje para el desarrollo web en entorno servidor

Características, requisitos, herramientas, comparativa y prueba de concepto

Grupo: Andrei y Rodrigo

Entrega: 21/09/2026



Contenido

1

El lenguaje Go

Origen y características

2

Requisitos de desarrollo

Qué instalar para programar

3

Requisitos de ejecución

Qué necesita el servidor

4

Servidores web

Compatibilidad y despliegue

5

IDEs y herramientas

Dónde se programa Go

6

Ejemplos de código

Cómo son los programas

7

Webs públicas

Quién usa Go en producción

8

Comparativa

Go frente a otras tecnologías

9

Prueba de concepto

Guía paso a paso: GoTareas

¿Qué es Go?

Go (o Golang) es un lenguaje de código abierto creado en Google en 2007 por Robert Griesemer, Rob Pike y Ken Thompson, y publicado en 2009.

Compilado: genera un ejecutable nativo, sin intérprete ni máquina virtual.

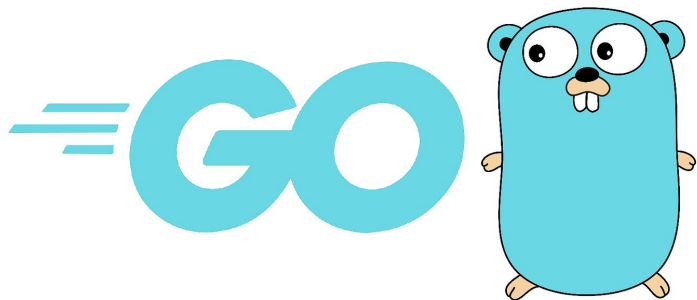
Tipado estático con inferencia de tipos (`x := 5`) y genéricos.

Recolector de basura: gestión automática de memoria.

Concurrencia integrada: goroutines y canales.

Sintaxis mínima: 25 palabras reservadas, formato único con gofmt.

Compatibilidad Go 1: el código de 2012 sigue compilando hoy.



2009

Presentación pública

2012

Versión estable Go 1.0

1.27

Última versión (ago. 2026)

¿Por qué Go en el servidor web?

net/http

Servidor HTTP/1.1, HTTP/2 y TLS en la biblioteca estándar. Desde Go 1.22 el router admite métodos y parámetros: "GET /tareas/{id}".

html/template

Motor de plantillas que escapa automáticamente el HTML según el contexto, protegiendo frente a XSS.

encoding/json

Serializa structs a JSON con etiquetas. Ideal para APIs REST y microservicios.

Goroutines

Cada petición se atiende en una goroutine (unos pocos KB), lo que permite miles de conexiones simultáneas.

embed

Plantillas, CSS e imágenes se pueden meter dentro del propio ejecutable.

Binario único

Compilación cruzada con GOOS/GOARCH: desde Windows se puede generar un ejecutable para Linux.

Requisitos para el desarrollo

- 1 Toolchain de Go**

Descarga oficial en go.dev/dl (instalador .msi, .pkg o tar.gz). Incluye compilador, gestor de módulos, tests, formateador y documentación.
- 2 Sistema operativo**

Windows 10 / Server 2016 o superior, macOS 13 Ventura o superior (exigido desde Go 1.27), Linux con kernel 3.2+, FreeBSD. Arquitecturas amd64, arm64 y otras.
- 3 Editor o IDE**

VS Code con la extensión oficial de Go o GoLand. Ambos usan gopls (servidor de lenguaje).
- 4 Git**

Los módulos de terceros se descargan desde repositorios Git (go get).
- 5 Opcionales**

Docker para empaquetar, curl o Postman para probar APIs, una base de datos (PostgreSQL, MySQL, SQLite).

comprobar la instalación

```
$ go version  
go version go1.27.1 windows/amd64
```

```
$ go env GOPATH GOOS GOARCH  
C:\Users\alumno\go  
windows  
amd64
```

```
$ mkdir miweb && cd miweb  
$ go mod init miweb  
go: creating new go.mod: module  
miweb
```

Requisitos para la ejecución de las páginas

PHP, Java, Python, Node.js

Código fuente

Intérprete / JVM / runtime

Servidor (Apache, Tomcat...)

Dependencias instaladas

Go

Un único ejecutable

Contiene el servidor HTTP, el runtime, las librerías y (con embed) las plantillas y archivos estáticos.

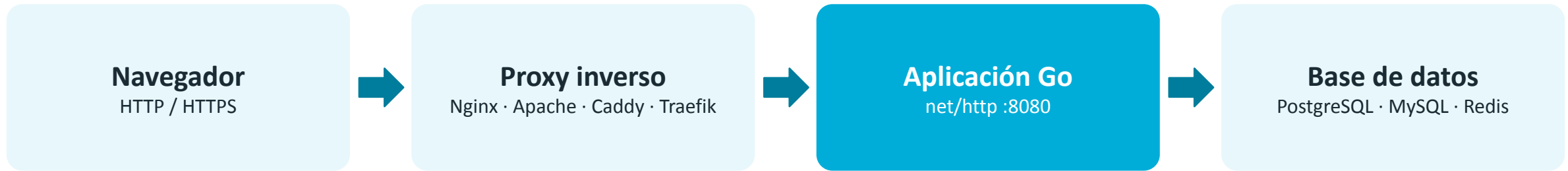
En el servidor: solo el binario compilado para su SO y arquitectura. No hace falta instalar Go.

Enlazado estático (CGO_ENABLED=0): funciona incluso en imágenes Docker vacías (scratch o distroless).

Configuración: variables de entorno (PORT, cadenas de conexión) y permisos para abrir el puerto.

En el cliente: un navegador cualquiera; recibe HTML, CSS, JS o JSON estándar.

Compatibilidad con servidores web



Servidor integrado

La aplicación Go ES el servidor web. `net/http` soporta HTTP/1.1, HTTP/2 y TLS (`ListenAndServeTLS`). En Go 1.27 el servidor HTTP/2 atiende señales de prioridad del cliente (RFC 9218).

Detrás de un proxy

Lo habitual en producción: Nginx (`proxy_pass`), Apache (`mod_proxy`), IIS (ARR) o Caddy y Traefik, que están escritos en Go y gestionan HTTPS automático con Let's Encrypt.

Otras formas

CGI y FastCGI con `net/http/cgi` y `net/http/fcgi`. Contenedores Docker y Kubernetes. Nube: Google Cloud Run, AWS Lambda, Azure Container Apps, Fly.io, Render.

IDEs y herramientas de desarrollo

VS Code + extensión Go

Gratis

Gratuito. Extensión oficial del equipo de Go: autocompletado, depuración, tests y refactorización mediante gopls.

GoLand (JetBrains)

Licencia educativa gratis

IDE específico para Go. Depurador, profiler, soporte de bases de datos y HTTP client. La versión 2026.2 ya soporta Go 1.27.

Vim / Neovim, Zed, Sublime, Emacs

Gratis

Editores ligeros que se integran con gopls vía LSP.

Otras útiles: staticcheck y golangci-lint (linters), air (recarga en caliente), sqlc (genera código a partir de SQL).

Herramientas del comando go

`go run`

compilar y ejecutar

`go build`

generar ejecutable

`go test`

tests y benchmarks

`gofmt`

formato estándar

`go vet`

análisis estático

`go mod`

dependencias

`go fix`

modernizar código

`dlv`

Delve, depurador (externo)

Frameworks y librerías del ecosistema

Routers y frameworks

- net/http (estándar)
- Gin: el más popular
- Echo: minimalista, con middlewares
- Fiber: API parecida a Express
- Chi: compatible con net/http

Vistas y plantillas

- html/template (estándar)
- templ: componentes tipados
- htmx: interactividad sin JS
- Hugo: generador de sitios estáticos

Datos y otros

- database/sql (estándar)
- GORM: ORM
- sqlc: SQL a código Go
- gRPC: microservicios
- Buffalo / Beego: full-stack

La filosofía de Go favorece la biblioteca estándar: muchos proyectos no usan ningún framework.

Ejemplo 1: «Hola mundo» web

```
main.go
package main

import (
    "fmt"
    "net/http"
)

func inicio(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, ";Hola desde Go!")
}

func saludo(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hola, %s", r.PathValue("nombre"))
}

func main() {
    http.HandleFunc("GET /", inicio)
    http.HandleFunc("GET /saludo/{nombre}", saludo)
    http.ListenAndServe(":8080", nil)
}
```

package main indica que se genera un ejecutable.

HandleFunc asocia una ruta (con método HTTP) a una función manejadora.

w es la respuesta; **r** es la petición.

{nombre} es un parámetro de ruta que se lee con **PathValue**.

ListenAndServe arranca el servidor en el puerto 8080.

```
$ go run main.go
$ curl localhost:8080/saludo/Ana
Hola, Ana
```

Ejemplo 2: página dinámica con plantilla

```
main.go (fragmento)

type Producto struct {
    Nombre string
    Precio float64
}

var tmpl = template.Must(
    template.ParseFiles("lista.html"))

func lista(w http.ResponseWriter,
           r *http.Request) {
    datos := []Producto{
        {"Teclado", 29.9},
        {"Ratón", 15.5},
    }
    tmpl.Execute(w, datos)
}
```

```
lista.html

<h1>Catálogo</h1>
<ul>
    {{range .}}
        <li>{{.Nombre}}:
            {{printf "%.2f" .Precio}} €</li>
    {{else}}
        <li>Sin productos</li>
    {{end}}
</ul>
```

Sintaxis de plantillas

{{.Campo}} imprime un dato, {{range}} recorre listas, {{if}} condiciona y {{template}} incluye otras plantillas. html/template escapa todo automáticamente: si Nombre contiene <script>, se muestra como texto y no se ejecuta.

Ejemplo 3: API REST con el framework Gin

```
main.go
package main

import ("net/http"; "github.com/gin-gonic/gin")

type Alumno struct {
    ID      int    `json:"id"`
    Nombre string `json:"nombre" binding:"required"`
}

var alumnos = []Alumno{{1, "Lucía"}, {2, "Marcos"}}

func main() {
    r := gin.Default()
    r.GET("/alumnos", func(c *gin.Context) {
        c.JSON(http.StatusOK, alumnos)
    })
    r.POST("/alumnos", func(c *gin.Context) {
        var a Alumno
        if err := c.ShouldBindJSON(&a); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
            return
        }
        a.ID = len(alumnos) + 1; alumnos = append(alumnos, a)
        c.JSON(http.StatusCreated, a)
    })
    r.Run(":8080")
}
```

```
$ go get github.com/gin-gonic/gin
```

```
$ curl localhost:8080/alumnos
[{"id":1,"nombre":"Lucía"},
 {"id":2,"nombre":"Marcos"}]
```

Etiquetas de struct (``json:"..."``) definen cómo se serializa a JSON.

binding:"required" valida automáticamente la entrada.

gin.Default() añade log y recuperación ante errores.

go get descarga la dependencia y la anota en go.mod.

Webs y servicios públicos hechos con Go



go.dev / pkg.go.dev

La web oficial del lenguaje y su buscador de paquetes están escritos en Go.



Twitch

Chat y servicios de vídeo en directo con millones de conexiones concurrentes.



Uber

Cientos de microservicios de backend, entre ellos el de geolocalización.



Cloudflare

Gran parte de su infraestructura de red y APIs.



Grafana / Gitea / Mattermost

Aplicaciones web completas cuyo backend es Go (grafana.com, gitea.com).



Docker Hub y Kubernetes

Docker y Kubernetes están escritos en Go; kubernetes.io y letsencrypt.org se generan con Hugo.

Otras empresas que usan Go: Google (YouTube, Google Cloud), Dropbox, PayPal, American Express, Netflix, SoundCloud, Monzo, Mercado Libre. Casos de estudio en go.dev/solutions.

Comparativa con otras tecnologías

	Go	PHP (Laravel)	Node.js	Python (Django)	Java (Jakarta)	ASP.NET Core
Tipado	Estático	Dinámico	Dinámico (TS: estático)	Dinámico	Estático	Estático (C#)
Ejecución	Binario nativo	Intérprete + OPcache	V8 (JIT)	Intérprete	JVM (JIT)	CLR (JIT/AOT)
Servidor	Integrado	Apache/Nginx + FPM	Integrado	Gunicorn/uvicorn	Tomcat, WildFly	Kestrel integrado
Concurrencia	Goroutines	Proceso por petición	Bucle de eventos	Async / workers	Hilos (virtuales)	async/await
Rendimiento	Muy alto	Medio	Alto	Medio	Muy alto	Muy alto
Memoria	Baja	Baja-media	Media	Media	Alta	Media
Despliegue	Copiar 1 fichero	Subir fuentes	node_modules	venv + deps	WAR/JAR + JVM	Publicar + runtime
Aprendizaje	Fácil	Fácil	Fácil	Muy fácil	Difícil	Media
Hosting barato	Medio	Excelente	Bueno	Bueno	Medio	Medio

Valoraciones orientativas basadas en benchmarks públicos (p. ej. TechEmpower) y en la documentación de cada tecnología.

Ventajas e inconvenientes de Go



Ventajas

- Rendimiento cercano a Java/C# con mucho menos consumo de memoria
- Concurrencia sencilla con goroutines y canales
- Despliegue trivial: un solo ejecutable, imágenes Docker de pocos MB
- Compilación muy rápida y biblioteca estándar completa
- Código uniforme y legible gracias a gofmt
- Estabilidad: promesa de compatibilidad Go 1



Inconvenientes

- Ecosistema web menos maduro que Laravel, Django o Spring
- Pocos frameworks «todo incluido» (autenticación, admin, ORM)
- Gestión de errores verbosa (if err != nil)
- El hosting compartido barato suele estar pensado para PHP
- Menos adecuado para webs de contenido tipo CMS (WordPress)
- Menos ofertas de empleo junior que PHP o Java en España

GoTareas

Una aplicación web de lista de tareas hecha solo con la biblioteca estándar de Go.

- ✓ HTML generado en el servidor con html/template
- ✓ Formularios con el patrón POST-Redirect-GET
- ✓ API REST que devuelve JSON
- ✓ CSS embebido en el ejecutable (embed)
- ✓ Middleware de registro de peticiones
- ✓ Concurrencia segura con sync.Mutex
- ✓ Despliegue con Docker y Nginx



Resultado final ejecutándose en <http://localhost:8080>

Código completo en el anexo [gotareas-poc.zip](#) · probado con Go 1.22+

Pasos 1 y 2: instalar Go y crear el proyecto

1 Instalar Go

Windows/macOS: descargar el instalador de go.dev/dl y ejecutarlo.

Linux: descomprimir en `/usr/local` y añadir `/usr/local/go/bin` al `PATH`.

Comprobar: abrir una terminal nueva y ejecutar `go version`.

VS Code: instalar la extensión «Go» y aceptar la instalación de `gopls`.

```
# Linux
$ wget https://go.dev/dl/go1.27.1.linux-amd64.tar.gz
$ sudo tar -C /usr/local -xzf go1.27.1.linux-amd64.tar.gz
$ echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
```

2 Crear el módulo y la estructura

```
$ mkdir gotareas && cd gotareas
$ go mod init gotareas
$ mkdir templates static
```

estructura del proyecto

```
gotareas/
├── go.mod           ← definición del módulo
├── main.go         ← servidor, rutas y handlers
├── templates/
│   └── index.html  ← plantilla de la página
├── static/
│   └── estilo.css  ← hoja de estilos
├── Dockerfile      ← imagen de contenedor
└── nginx.conf      ← proxy inverso (opcional)
```

go.mod solo contiene «module gotareas» y la versión de Go: no hay dependencias externas.

Paso 3: el modelo y el almacén de datos

```
main.go · modelo
// Tarea es el modelo de datos de la aplicación.
type Tarea struct {
    ID      int    `json:"id"`
    Titulo  string `json:"titulo"`
    Hecha   bool   `json:"hecha"`
}

// Almacen guarda las tareas en memoria de forma
// segura entre goroutines.
type Almacen struct {
    mu      sync.Mutex
    tareas []Tarea
    sigID   int
}

func (a *Almacen) Crear(titulo string) Tarea {
    a.mu.Lock()
    defer a.mu.Unlock()
    a.sigID++
    t := Tarea{ID: a.sigID, Titulo: titulo}
    a.tareas = append(a.tareas, t)
    return t
}
```

¿Por qué un Mutex?

Cada petición HTTP se ejecuta en su propia goroutine. Sin el candado, dos peticiones podrían modificar el slice a la vez.

Métodos

Listar() devuelve una copia del slice y Alternar(id) cambia el estado Hecha. Siguen el mismo patrón Lock / defer Unlock.

Ampliación posible

Sustituir el almacén en memoria por SQLite o PostgreSQL con database/sql sin cambiar los handlers.

Paso 4: servidor, rutas y archivos embebidos

```
main.go · arranque
//go:embed templates/*.html
var plantillas embed.FS

//go:embed static
var estaticos embed.FS

var (
    almacen = &Almacen{}
    tmpl = template.Must(template.ParseFS(plantillas, "templates/*.html"))
)

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /{$}", paginaInicio)
    mux.HandleFunc("POST /tareas", crearTarea)
    mux.HandleFunc("POST /tareas/{id}/alternar", alternarTarea)
    mux.HandleFunc("GET /api/tareas", apiListar)
    mux.Handle("GET /static/", http.FileServerFS(estaticos))

    puerto := os.Getenv("PORT")
    if puerto == "" { puerto = "8080" }
    log.Printf("Servidor en http://localhost:%s", puerto)
    log.Fatal(http.ListenAndServe(":"+puerto, registrar(mux)))
}
```

//go:embed

mete las plantillas y el CSS dentro del ejecutable.

GET /{\$}

solo la raíz exacta; el resto de rutas dan 404.

{id}

parámetro de ruta, leído con `r.PathValue("id")`.

PORT

variable de entorno: así funciona en Cloud Run o Render.

registrar(mux)

middleware que envuelve todo el router.

Paso 5: los handlers y el middleware

main.go · páginas HTML

```
func paginaInicio(w http.ResponseWriter, r *http.Request) {
    datos := map[string]any{"Tareas": almacen.Listar()}
    if err := tpl.ExecuteTemplate(w, "index.html", datos); err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
    }
}

func crearTarea(w http.ResponseWriter, r *http.Request) {
    if titulo := r.FormValue("titulo"); titulo != "" {
        almacen.Crear(titulo)
    }
    http.Redirect(w, r, "/", http.StatusSeeOther) // PRG
}

func alternarTarea(w http.ResponseWriter, r *http.Request) {
    id, err := strconv.Atoi(r.PathValue("id"))
    if err != nil || !almacen.Alternar(id) {
        http.NotFound(w, r)
        return
    }
    http.Redirect(w, r, "/", http.StatusSeeOther)
}
```

main.go · API JSON

```
func apiListar(w http.ResponseWriter,
    r *http.Request) {
    w.Header().Set("Content-Type",
        "application/json")
    json.NewEncoder(w).Encode(almacen.Listar())
}
```

main.go · middleware

```
func registrar(next http.Handler) http.Handler {
    return http.HandlerFunc(func(
        w http.ResponseWriter, r *http.Request) {
        log.Printf("%s %s", r.Method, r.URL.Path)
        next.ServeHTTP(w, r)
    })
}
```

Un middleware es una función que recibe un handler y devuelve otro: sirve para logs, autenticación o CORS.

Paso 6: la plantilla HTML

```
templates/index.html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>GoTareas</title>
  <link rel="stylesheet" href="/static/estilo.css">
</head>
<body>
  <h1>GoTareas</h1>
  <form method="post" action="/tareas">
    <input name="titulo" placeholder="Nueva tarea" required>
    <button>Añadir</button>
  </form>
  <ul>
    {{range .Tareas}}
    <li class="{{if .Hecha}}hecha{{end}}">
      <form method="post" action="/tareas/{{.ID}}/alternar">
        <button>{{if .Hecha}}0{{else}}✓{{end}}</button>
      </form>
      {{.Titulo}}
    </li>
    {{else}}<li>No hay tareas todavía.</li>{{end}}
  </ul>
  <p>Total: {{len .Tareas}} · <a href="/api/tareas">Ver JSON</a></p>
</body>
</html>
```

{{range .Tareas}} repite el por cada tarea; **{{else}}** se muestra si la lista está vacía.

{{if .Hecha}} añade la clase CSS que tacha la tarea.

{{len .Tareas}} función integrada de las plantillas.

Formularios HTML normales: la app funciona sin JavaScript.

estilo.css se sirve desde /static/ (está en el anexo).

Paso 7: ejecutar y probar

```
terminal 1 · servidor
$ go run .
2026/09/21 10:00:00 Servidor en http://localhost:8080
2026/09/21 10:00:05 GET /
2026/09/21 10:00:05 GET /static/estilo.css
2026/09/21 10:00:09 POST /tareas
```

```
terminal 2 · pruebas
$ curl localhost:8080/api/tareas
[{"id":1,"titulo":"Instalar Go","hecha":false},
 {"id":2,"titulo":"Preparar la presentación",...}]

$ curl -X POST -d "titulo=Probar la API" localhost:8080/tareas
$ curl -X POST localhost:8080/tareas/1/alternar

$ curl localhost:8080/api/tareas
[{"id":1,"titulo":"Instalar Go","hecha":true}, ...
 {"id":3,"titulo":"Probar la API","hecha":false}]
```

GoTareas

Añadir

Instalar Go



Preparar la presentación



Presentar en clase el 21/09

Total: 3 · [Ver JSON](#)

Checklist para la demo en clase

- Abrir <http://localhost:8080> y añadir una tarea
- Marcarla como hecha y ver el tachado
- Abrir </api/tareas> y mostrar el JSON
- Enseñar el log de peticiones en la terminal
- Probar </xyz> y ver el 404
- Mostrar el tamaño del ejecutable (≈10 MB)

Paso 8: compilar y desplegar

A · binario

```
# Ejecutable para el sistema actual
$ go build -o gotareas .

# Compilación cruzada: desde Windows para Linux
PS> $env:GOOS="linux"; $env:GOARCH="amd64"
PS> go build -o gotareas-linux .

# En el servidor Linux solo hace falta el binario
$ scp gotareas-linux usuario@servidor:/opt/gotareas/
$ PORT=8080 ./gotareas-linux
```

B · Dockerfile (multi-etapa)

```
# Etapa 1: compilar
FROM golang:1.27 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /gotareas .

# Etapa 2: imagen mínima
FROM gcr.io/distroless/static
COPY --from=build /gotareas /gotareas
EXPOSE 8080
ENTRYPOINT ["/gotareas"]
```

C · Nginx como proxy inverso

```
server {
    listen 80;
    server_name gotareas.local;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
    }
}
```

```
$ docker build -t gotareas .
$ docker run -p 8080:8080 gotareas
```

La imagen final ocupa unos pocos MB porque solo contiene el ejecutable, frente a los cientos de MB de una imagen con PHP + Apache o una JVM.

Posibles ampliaciones de la prueba

1 Persistencia

Guardar las tareas en SQLite (modernc.org/sqlite, sin CGO) o PostgreSQL con database/sql.

2 API REST completa

Añadir POST, PUT y DELETE en /api/tareas con JSON de entrada.

3 Interactividad

Usar htmx para actualizar la lista sin recargar la página.

4 Tests

Probar los handlers con net/http/httptest y go test.

5 Autenticación

Sesiones con cookies seguras o JWT mediante un middleware.

6 Framework

Reescribir las rutas con Gin o Echo y comparar el código.

```
ejemplo de test · main_test.go
func TestApiListar(t *testing.T) {
    rec := httptest.NewRecorder()
    apiListar(rec, httptest.NewRequest("GET", "/api/tareas", nil))
    if rec.Code != 200 { t.Fatalf("código %d", rec.Code) }
}
```

Conclusiones

1

Simple y productivo

Con la biblioteca estándar se construye una web completa sin frameworks ni dependencias.

2

Pensado para servidores

Concurrencia, rendimiento y bajo consumo lo hacen ideal para APIs y microservicios.

3

Despliegue sencillo

Un ejecutable sin runtime: se copia, se ejecuta o se mete en un contenedor mínimo.

4

Menos adecuado para...

Webs de contenidos o proyectos que se benefician de un CMS o de un framework con todo incluido.

¿Preguntas?

Referencias

Sitio oficial de Go

<https://go.dev>

Notas de la versión 1.27

<https://go.dev/doc/go1.27>

Paquete net/http

<https://pkg.go.dev/net/http>

Casos de uso de Go

<https://go.dev/solutions/>

Framework Gin

<https://gin-gonic.com>

GoLand (JetBrains)

<https://www.jetbrains.com/go/>

Descargas e instalación

<https://go.dev/doc/install>

Tutorial oficial: aplicaciones web

<https://go.dev/doc/articles/wiki/>

Paquete html/template

<https://pkg.go.dev/html/template>

Go by Example

<https://gobyexample.com>

Extensión Go para VS Code

<https://marketplace.visualstudio.com/items?itemName=golang.go>

Wikipedia: Go

[https://es.wikipedia.org/wiki/Go_\(lenguaje_de_programación\)](https://es.wikipedia.org/wiki/Go_(lenguaje_de_programación))

Anexo: código fuente de la prueba de concepto en gotareas-poc.zip